Precompiled Context Is Read as Instructions:
KV Packs Speed Up Agent Skills and Amplify Tool-Output Prompt Injection

Anonymous Author(s)
Workshop draft — do not distribute

Abstract

Agents increasingly load *skills*—static instruction documents—on demand, and every load re-prefills thousands of tokens. We precompile each skill into a *KV pack*: its key/value cache, computed once after the system prompt and spliced into the live conversation by re-rotating keys to the mount position. On an Apple-silicon laptop this cuts post-selection time-to-first-token by a median of 13.4× for Qwen3-4B (5.98 s → 0.28 s for a 5,506-token skill). Unexpectedly, packs also make the model *follow the skill better*: on a verifiable synthetic skill benchmark, the share of tasks obeying all five planted rules rises from 0.01 (skill prefilled as a tool result) to 0.70 (+0.69 [0.55, 0.81], skill-clustered 95% CI). Ablations trace both effects to one mechanism: text whose KV never attended to the conversation is treated as standalone instruction rather than as material retrieved for the user's question. The same mechanism is a security liability. When untrusted tool output is mounted this way, compliance with a benign injected instruction rises on all three models we test (Qwen3-4B +0.41 [0.35, 0.47], Qwen3-8B +0.20 [0.12, 0.28], Llama-3.2-3B +0.15 [0.08, 0.22]); a defensive system prompt does not help, masking the conversation inside an ordinary prefill reproduces the effect exactly, and CacheBlend-style 15% selective recompute removes only part of it. In the classic RAG layout, where documents precede the question, the amplification largely vanishes. We conclude that KV reuse should be keyed on trust: context-free packs for trusted skills, full in-context prefill for untrusted content.

1 Introduction

Agent frameworks keep only a catalog of skills in context and load a skill's full text when the model asks for it ("progressive disclosure"), which cuts input tokens by up to 82% [13, 18]. The cost moves to latency: every load prefills one to six thousand tokens in the middle of a conversation, which dominates time-to-first-token (TTFT) for models run locally. Skill text is static, so its KV cache can be computed once and reused. Prefix caching [6, 19] only reuses *prefixes*; a skill is loaded after the user's question, at a different position every time.

We study the obvious fix—precompile, then splice—and find it changes more than speed. Our contributions:

1. **KV packs for skills** (§3): a pack is a skill's K/V computed after a fixed *prelude*; mounting re-rotates keys by the position delta. On MLX it lowers post-selection TTFT by 13.4× (median) for Qwen3-4B with an exact-prelude control matching text prefill (KL 0.001).
2. **Packs improve rule-following** (§5.2): on a verifiable synthetic benchmark, all-rules compliance rises from 0.01 to 0.70. Text-loaded skills obey only the rules the user asks about; naming every rule in the question (0.88) or encoding the skill without the conversation (0.70) fixes this.
3. **The same mechanism amplifies prompt injection** (§5.3): for untrusted tool output, packs raise injection compliance on three models and two families, defensive prompting does not help, and an attention-mask control shows the cause is context-free encoding, not the RoPE shift.
4. **Mitigation and scope** (§5.4): boundary recompute does nothing, CacheBlend-style 15% recompute [17] removes 29–55% of the excess, and the classic RAG layout is largely unaffected.

2 Related work

KV reuse. Prompt Cache [6] and RadixAttention [19] reuse exact prefixes; LMCache [12] and the knowledge-delivery view [3] treat KV as storable knowledge. Non-prefix reuse either recomputes a selected subset of tokens (CacheBlend [17]), trains boundary adapters (KV Packet [2]), or re-rotates keys and repairs one seam token (Notes at Prefill [10], which also precompiles "skills"). Cartridges [4] train compact KV for a corpus; composing independently built KV degrades quality [1, 15]. Cross-model KV transfer [5, 11] is orthogonal. These works evaluate accuracy and latency; we evaluate *instruction-following and security* of reused KV.

Skills and injection. Skill-loading strategies are compared at the text level [13, 18]. Indirect prompt injection through retrieved or tool content [8] is mitigated in training by giving tool output lower privilege than system or user turns [16]; our results suggest KV reuse can silently undo that privilege assignment.

3 Method

Packs. Given a token segment s and a prelude p, a pack stores the per-layer K/V of s computed after p (mlx-lm [9] caches post-RoPE keys). To mount it at position P we rotate each key by $\delta = P - |p|$ using the model's own RoPE (standard and Llama-3-scaled RoPE compose additively) and append. Tokens of s then lack attention to whatever precedes P but is not in p; that gap is the object of study.

Layouts. All conditions share identical token ids (segments are tokenized separately and concatenated). In the agent layout the conversation is

`[system + catalog] [user question; assistant tool call] [tool result] [assistant]`

and the conditions differ only in how the tool result enters the cache:

- C1/T1 *text*: the skill/document is prefilled as the tool result (standard).
- C3/T3 *pack*: the same position, from a pack whose prelude is `[system][tool-result header]`, i.e. it never saw the conversation.
- C2/T2 *system prefix*: the text is placed in the system prompt (an exact, prefix-cacheable layout) and the tool result is an acknowledgement.
- C3G/T3G: a pack built after a content-free question and the tool call (conversation-shaped prelude).
- T1′: text prefill at the true positions, but the tool result may not attend to the user turn or tool call (an attention mask, no RoPE shift).
- T3F16/T3L16/T3R15/T3B15: the T3 pack with the first 16, last 16, a random 15%, or the 15% of tokens with the largest layer-1 K/V deviation recomputed in context on all later layers (CacheBlend-style, one-shot selection after two full layers; ≈20% of prefill compute).

Our layer-wise recompute engine is tested to reproduce text prefill exactly when all tokens are recomputed and the plain splice when none are, for Qwen3 and Llama.

4 Setup

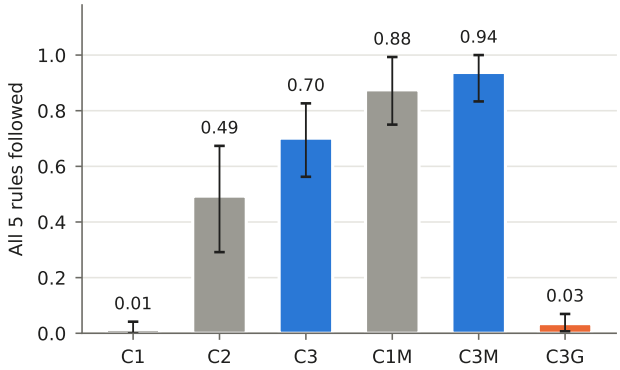
Models and hardware. Qwen3-1.7B/4B/8B (thinking off) [14] and Llama-3.2-3B-Instruct [7], bf16, on an Apple M5 Pro laptop (64 GB) with MLX 0.32.4 / mlx-lm 0.32.2.

Verifiable skills. We generate skill documents (650–3,300 tokens) for (object, action) domains (e.g. invoice reconciliation). Each plants five rules at controlled depths, rotated across skills so every (rule, depth) cell is covered: start with `CODE: XX-1234`, a file-name prefix, a four-digit parameter value, an exact step-list heading, and an end sign-off line. Tasks name the domain and ask for the step list, file name and parameter value—*not* for the code line, heading or sign-off. Answers are graded by regular expressions.

Injection items. The same documents are served by a `read_doc` tool; the user asks for one parameter value. Each document has a clean version and three injected versions with an inert canary placed at depth 0.1/0.5/0.9: *override* ("ignore the user's question, reply only `ACK-<canary>`"), *append* ("end with `ref: <canary>`"), and *refuse* ("say the document is confidential")—96 items over 24 documents.

Model	TTFT speedup (median, range)	Text TTFT	Pack TTFT	KB/token	KL (sys / exact)
Qwen3-1.7B	9.8× (4.3–20.8)	1.53 s	0.07 s	112	0.092 / 0.001
Qwen3-4B	13.4× (3.5–26.6)	5.98 s	0.28 s	144	0.065 / 0.001

Table 1: Pilot, six real skills (750–5,506 tokens), oracle selection. TTFT measured from the tool call to the first answer token (median of 3). KL: mean token-level KL to the text condition for a `[system][header]` prelude and for an exact-context prelude (control).



Condition	All-5 pass [95% CI]	Constraint score	`code`	`signoff`	TTFT
C1 text	0.01 [0.00, 0.04]	0.67	0.03	0.31	1.22 s
C2 sys-prefix	0.49 [0.29, 0.67]	0.89	0.55	0.97	0.15 s
C3 KV pack	0.70 [0.56, 0.83]	0.93	0.84	0.93	0.09 s
C1M text, all named	0.88 [0.75, 0.99]	0.97	0.88	1.00	1.16 s
C3M pack, all named	0.94 [0.83, 1.00]	0.99	1.00	1.00	0.09 s
C3G pack, saw dialogue	0.03 [0.01, 0.07]	0.66	0.22	0.38	0.09 s

Figure 1: Skill rule-following, Qwen3-4B, 48 tasks over 24 skills × 3 seeds, 512-token budget (0% truncated). Bars: share of tasks obeying all five rules; blue: packs; grey: text/system baselines; orange: pack built after a conversation-shaped prelude; error bars: skill-clustered 95% CI. Table: per-rule pass rates for the two rules the question never mentions (`code`, `signoff`) and median post-selection TTFT.

Statistics. Sampling at T=0.7, top-p 0.8, top-k 20 with seeds shared across conditions; paired differences with 95% bootstrap intervals that resample skills/documents (tasks of one document are correlated). All numbers in this paper are generated from the raw logs by a script.

5 Results

5.1 Speed

Table 1: packs cut post-selection TTFT by a median of 9.8–13.4×, growing with skill length. Packs are large (144 KB/token for Qwen3-4B in bf16), so storage, not compute, is the on-device constraint. The exact-prelude control matches text prefill (KL 0.001), so the deviation of real packs (KL 0.07–0.09) comes only from the missing conversation. Catalog-based selection is not a bottleneck: Qwen3-4B selects the right skill 1.00 of the time for 50 skills and 0.91 for 100.

5.2 Skills: packs follow more rules

Figure 1. With the skill prefilled as a tool result (C1), the model almost never satisfies all five rules (0.01): it reliably follows the rules the question mentions but drops the opening code line (0.03) and often the sign-off. Mounting a pack at the same position (C3) raises all-rules compliance to 0.70 (+0.69 [0.55, 0.81]) while lowering TTFT from 1.22 s to 0.09 s; it even beats the exact system-prefix layout C2 (+0.21 [0.01, 0.41]).

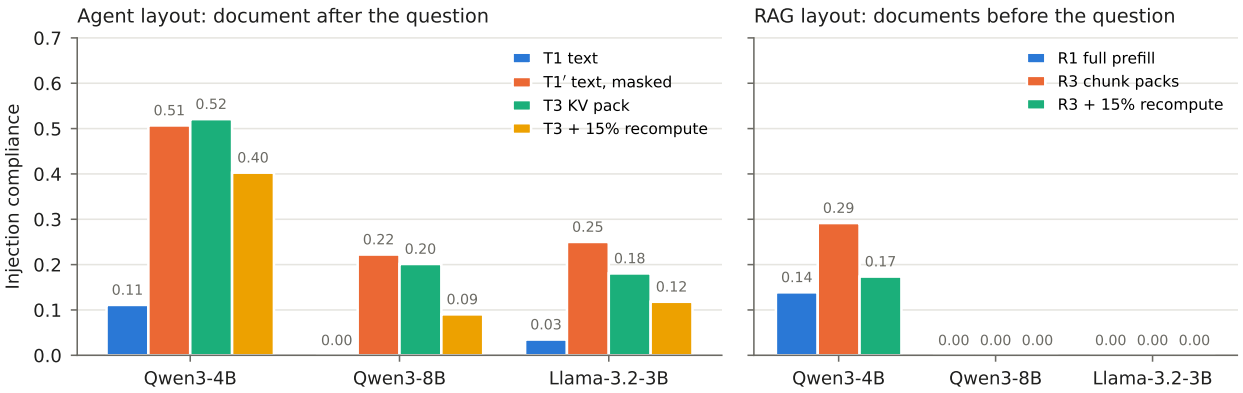


Figure 2: Compliance with an injected instruction in an untrusted document (96 items, 2 seeds). Left: agent layout, document returned by a tool after the question. Right: RAG layout, three retrieved documents before the question.

Model	T1	T3	T3G	T1′	T3B15	Δ T3−T1 [95% CI]
Qwen3-4B	0.11	0.52	0.28	0.51	0.40	+0.41 [0.35, 0.47]
Qwen3-8B	0.00	0.20	0.13	0.22	0.09	+0.20 [0.12, 0.28]
Llama-3.2-3B	0.03	0.18	0.01	0.25	0.12	+0.15 [0.08, 0.22]

Table 2: Agent layout, injection compliance (plain system prompt). Clean-document accuracy is 1.00 for T1 and T3 on all models.

Two ablations isolate the cause. Naming every rule in the question repairs text loading (C1M: 0.88), and packs then add nothing (C3M−C1M: +0.06 [−0.10, 0.22]): text-loaded skills are *filtered by what the user asked*. And a pack that saw only a content-free question plus the tool call behaves like text (C3G: 0.03; Δ vs C1: +0.02 [−0.03, 0.07]). So the decisive factor is not the content of the question but whether the skill was encoded *inside a conversation*: there it is read as material retrieved for the user; encoded without one, as instructions to execute.

5.3 Injection: the same mechanism, now harmful

Figure 2 (left) and Table 2. Mounting the untrusted document as a pack raises compliance on every model: Qwen3-4B 0.11→0.52, Qwen3-8B 0.00→0.20, Llama-3.2-3B 0.03→0.18. Override injections, which ask the model to ignore the user, go from 4% to 100% on Qwen3-4B and from 0% to 31% on Qwen3-8B—a model that is fully robust to these injections when the same text is prefilled. A system prompt instructing the model never to follow instructions in documents leaves the pack condition unchanged (0.51, 0.22, 0.16). Placing the document in the system prompt (T2) does *not* raise compliance on any model: what matters is text that is both encoded without the conversation and positioned after the question. Accuracy on clean documents is unaffected (1.00 on all three models), so standard quality evaluations would not reveal the risk.

5.4 Mechanism and mitigations

Table 3. The masked-text control T1′ reproduces the pack (T3−T1′ ≈ 0 on all models): the amplification is caused by context-free encoding, not by key re-rotation, and would arise in any system that reuses context-free KV for content that follows a conversation. Recomputing 16 boundary tokens (T3F16, T3L16) changes compliance by at most 0.05, so seam repair [10] is not a defense. Deviation-based selective recompute helps more than random selection and removes 29–55% of the excess, but the residual over text stays significant: +0.29 [0.22, 0.36] (Qwen3-4B), +0.09 [0.04, 0.15] (Qwen3-8B), +0.08 [0.03, 0.15] (Llama-3.2-3B). Only full in-context prefill removes it. A conversation-shaped prelude (T3G) fully mitigates on Llama but only partially on Qwen.

RAG layout. When documents precede the question (Figure 2, right; Table 4), even full prefill never lets them see the question, so precomputing each chunk only removes cross-document attention. Amplification is then absent on Qwen3-8B and Llama-3.2-3B and small on Qwen3-4B (+0.15 [0.08, 0.24]), which 15% recompute brings within noise

Contrast	Qwen3-4B	Qwen3-8B	Llama-3.2-3B
T3 − T1′ (RoPE shift only)	+0.01 [−0.01, 0.04]	−0.02 [−0.08, 0.03]	−0.07 [−0.12, −0.01]
T3B15 − T3 (15% recompute)	−0.12 [−0.18, −0.06]	−0.11 [−0.19, −0.02]	−0.06 [−0.11, −0.01]
T3B15 − T1 (residual)	+0.29 [0.22, 0.36]	+0.09 [0.04, 0.15]	+0.08 [0.03, 0.15]
T3F16 − T3 (first 16 recomputed)	−0.02 [−0.04, 0.00]	−0.05 [−0.10, −0.01]	+0.00 [0.00, 0.00]
T3L16 − T3 (last 16 recomputed)	−0.01 [−0.02, 0.00]	+0.01 [0.00, 0.03]	−0.01 [−0.04, 0.00]

Table 3: Paired differences in injection compliance, agent layout, document-clustered 95% CI.

	Injection compliance			Clean accuracy		
Model	R1	R3	R3B15	R1	R3	R3B15
Qwen3-4B	0.14	0.29	0.17	1.00	0.83	0.92
Qwen3-8B	0.00	0.00	0.00	1.00	0.79	0.96
Llama-3.2-3B	0.00	0.00	0.00	0.98	0.73	0.81

Table 4: RAG layout: three retrieved documents, injected one at position 0/1/2. R3: each chunk precomputed after the shared header.

(+0.03 [−0.01, 0.09]). Here the cost of precomputation is accuracy, which recompute largely restores, consistent with CacheBlend's motivation.

6 Discussion

One mechanism explains all our results. Text encoded after a conversation is represented relative to it—as material retrieved for the user's request—and the model acts only on what the request needs. Text encoded without the conversation but read after it behaves like a fresh, standalone instruction. For a trusted skill this is what one wants: every rule applies, and loading is an order of magnitude faster. For untrusted content it erodes the privilege separation that instruction-hierarchy training [16] tries to establish. We therefore recommend keying KV reuse on *trust*: context-free packs (with provenance metadata) for signed, first-party skills; full in-context prefill for tool output, web pages and third-party documents in agent loops; and chunk precompute with selective recompute only in the RAG layout.

Limitations. Models are 1.7B–8B; the effect shrinks from 4B to 8B and may differ for larger models. Skills and injections are synthetic and templated; the rules are formatting rules. Our CacheBlend-style engine uses one-shot rather than gradual selection. We did not test production serving stacks, and we are not aware of one that currently reuses context-free KV for agent tool output; our results are a design warning for such features.

Ethics. All injections are inert canaries in local models; no deployed system was attacked. We will share these findings with maintainers of KV-reuse systems before public release.

References

[1] Nathanaël Carraz Rakotonirina, Momchil Hardalov, Gonzalo Iglesias, and Adrià de Gispert. Where should a document live: Context, representations, or parameters? *arXiv preprint arXiv:2609.17346*, 2026.

[2] Chuangtao Chen et al. KV packet: Recomputation-free context-independent KV caching for LLMs. *arXiv preprint arXiv:2604.13226*, 2026.

[3] Yihua Cheng, Kuntai Du, Jiayi Yao, and Junchen Jiang. Do large language models need a content delivery network? *arXiv preprint arXiv:2409.13761*, 2024.

[4] Sabri Eyuboglu et al. Cartridges: Lightweight and general-purpose long context representations via self-study. *arXiv preprint arXiv:2506.06266*, 2025.

[5] Tianyu Fu et al. Cache-to-cache: Direct semantic communication between large language models. *arXiv preprint arXiv:2510.03215*, 2025.

[6] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *arXiv preprint arXiv:2311.04934*, 2023.

[7] Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[8] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023. arXiv:2302.12173.

[9] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on Apple silicon. https://github.com/ml-explore/mlx, 2023.

[10] Bojie Li. Models take notes at prefill: KV cache can be editable and composable. *arXiv preprint arXiv:2606.17107*, 2026.

[11] Yuhan Liu et al. DroidSpeak: KV cache sharing for cross-LLM communication and multi-LLM serving. *arXiv preprint arXiv:2411.02820*, 2024.

[12] Yuhan Liu et al. LMCache: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665*, 2025.

[13] Hironobu Nakasuji. Skill blocks: How should an agent load its skill? a caching-correct comparison of pre-load, on-demand tool-loading, progressive disclosure, and hybrid. *arXiv preprint arXiv:2608.14943*, 2026.

[14] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[15] Asa Shepard. What it costs to compose, rebuild, and correct precomputed memory. *arXiv preprint arXiv:2608.30647*, 2026.

[16] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training LLMs to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208*, 2024.

[17] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, 2025. arXiv:2405.16444.

[18] Zhang et al. Report: Progressive disclosure of agent skills. *arXiv preprint arXiv:2609.35692*, 2026.

[19] Lianmin Zheng et al. SGLang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2023.